

M12 GRAPH TRAVERSAL

1. TUJUAN

Tujuan Instruksional Umum :

Mampu mengimplementasikan *Graph* Traversal untuk kasus sederhana.

Tujuan Instruksional Khusus :

1. Dapat menjelaskan konsep Graph Traversal Breadth-First Search (BFS) dan Depth First Search (DFS).
2. Dapat menunjukkan path Graph jika menggunakan BFS atau DFS.
3. Dapat menggunakan algoritma BFS atau DFS untuk kasus sederhana.

2. DURASI WAKTU

2 pertemuan x 4,5 jam

3. DASAR TEORI

Algoritma BFS

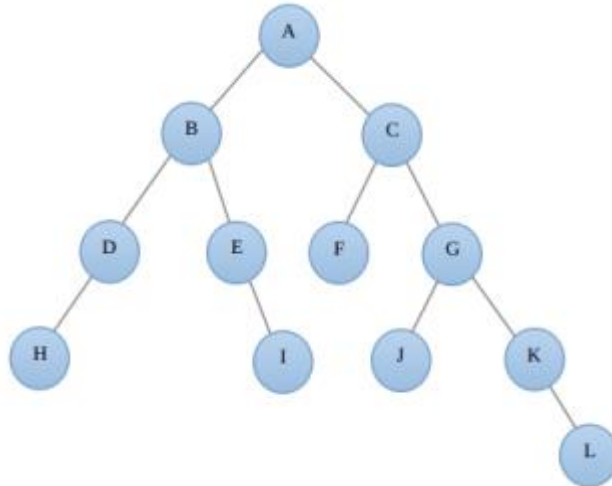
Pada algoritma BFS, pencarian dimulai dari simpul akar dan mengunjungi semua tetangga dari simpul tersebut. Kemudian, pencarian dilanjutkan dari setiap simpul terdekat ke simpul tetangga yang belum dikunjungi, dan seterusnya, sampai mendapatkan solusi.

- Traversal dimulai dari simpul v .
- Algoritma:
 1. Kunjungi simpul v ,
 2. Kunjungi semua simpul yang bertetangga dengan simpul v terlebih dahulu.

3. Kunjungi simpul yang belum dikunjungi dan bertetangga dengan simpul-simpul yang tadi dikunjungi, demikian seterusnya.

Contoh :

Jika terdapat Graph seperti dibawah ini:



Maka, urutan penelusurannya adalah : A – B – C – D – E – F – G – H – I – J – K – L

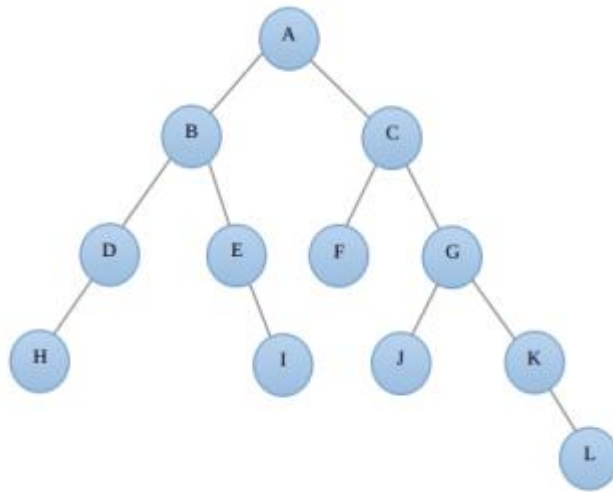
Algoritma DFS (Depth First Search)

Pada algoritma DFS, pencarian dimulai dari simpul akar, dilanjutkan dengan mengunjungi satu cabang sampai sedalam mungkin sebelum melakukan runut balik dan melanjutkan pencarian dari cabang lain.

- Traversal dimulai dari simpul v.
- Algoritma:
 1. Kunjungi simpul v,
 2. Kunjungi simpul w yang bertetangga dengan simpul v.
 3. Ulangi DFS mulai dari simpul w.

Contoh :

Jika terdapat Graph seperti dibawah ini:



Maka, urutan penelusurannya adalah : A – B – D – H – E – I – C – F – G – J – K – L

4. PERCOBAAN

PERCOBAAN 1 : Algoritma DFS (Rekursif)

1. Modifikasilah class Graph pada percobaan sebelumnya dengan menambahkan method dfs, source code tampak seperti berikut:

```
public void DFS(Vertex base) {
    if (base.isWasVisited()) {
        return;
    }
    base.setWasVisited(true);
    System.out.print(base.getLabel());
    for (int i = 0; i < numEdges; i++) {
        if (edges[i].getFromVertex() == base) {
            DFS(edges[i].getToVertex());
        } else if (edges[i].getToVertex() == base) {
            DFS(edges[i].getFromVertex());
        }
    }
}
```

2. Modifikasilah method main dalam program sehingga tampak seperti dibawah ini.

```
public static void main(String[] args) {
    Graph myGraph = new Graph();

    Vertex a = new Vertex("A");
    Vertex b = new Vertex("B");
    Vertex c = new Vertex("C");
    Vertex d = new Vertex("D");
    Vertex e = new Vertex("E");

    myGraph.addVertex(a);
    myGraph.addVertex(b);
    myGraph.addVertex(c);
    myGraph.addVertex(d);
    myGraph.addVertex(e);

    myGraph.addEdge(a, b);
    myGraph.addEdge(b, c);
    myGraph.addEdge(b, d);
    myGraph.addEdge(c, e);

    myGraph.displayAdjacent();

    myGraph.displayIncidence();

    System.out.println("");
}
```

```
myGraph.DFS(b);
}
```

3. Jalankan program, contoh output tampak seperti gambar dibawah:

Output

	Debugger Console		CobaGraph (run)		
	B	1	0	1	1
	C	0	1	0	0
	D	0	1	0	0
	E	0	0	1	0
		e0	e1	e2	e3
	A	1	0	0	0
	B	-1	1	1	0
	C	0	-1	0	1
	D	0	0	-1	0
	E	0	0	0	-1
	BACED				

PERCOBAAN 2 : Algoritma BFS

1. Modifikasilah class Graph pada percobaan sebelumnya dengan menambahkan method `public void BFS(Vertex base)`

```
public void BFS(Vertex base) {
    Queue<Vertex> q = new LinkedList();
    q.add(base);
    base.setWasVisited(true);
    while (!q.isEmpty()) {
        Vertex x = q.poll();
        System.out.print(x.getLabel());
        for (int i = 0; i < numEdges; i++) {
            if (edges[i].getFromVertex() == x &&
!edges[i].getToVertex().isWasVisited()) {
                q.add(edges[i].getToVertex());
                edges[i].getToVertex().setWasVisited(true);
            } else if (edges[i].getToVertex() == x &&
!edges[i].getFromVertex().isWasVisited()) {
                q.add(edges[i].getFromVertex());
                edges[i].getFromVertex().setWasVisited(true);
            }
        }
    }
}
```

2. Modifikasilah method main dalam program sehingga tampak seperti dibawah ini.

```
public static void main(String[] args) {  
    Graph myGraph = new Graph();  
  
    Vertex a = new Vertex("A");  
    Vertex b = new Vertex("B");  
    Vertex c = new Vertex("C");  
    Vertex d = new Vertex("D");  
    Vertex e = new Vertex("E");  
  
    myGraph.addVertex(a);  
    myGraph.addVertex(b);  
    myGraph.addVertex(c);  
    myGraph.addVertex(d);  
    myGraph.addVertex(e);  
  
    myGraph.addEdge(a, b);  
    myGraph.addEdge(b, c);  
    myGraph.addEdge(b, d);  
    myGraph.addEdge(c, e);  
  
    myGraph.displayAdjacent();  
  
    myGraph.displayIncidence();  
  
    myGraph.BFS(b);  
    System.out.println("");  
}
```

3. Jalankan program, contoh output tampak seperti gambar dibawah:

Output

Debugger Console		CobaGraph (run)			
B	1	0	1	1	0
C	0	1	0	0	1
D	0	1	0	0	0
E	0	0	1	0	0
	e0	e1	e2	e3	
A	1	0	0	0	
B	-1	1	1	0	
C	0	-1	0	1	
D	0	0	-1	0	
E	0	0	0	-1	

BACDE
BUILD SUCCESSFUL (total time: 0 seconds)

5. LATIHAN

Modifikasi program diatas agar dapat menentukan jalur terpendek dari vertex A ke E dari gambar graph dibawah ini:

